Watchdog Daemon Implementation Overview

Host-side WiFi stick recovery service

Codex

June 7, 2026

Table of Contents

What Problem The Watchdog Solves **3**

Files Involved **3**

High-Level Architecture **4**

Health Model **5**

How Health Is Read **6**
- Interface Presence . . . 6
- IPv4 Address . . . 6
- WPA State . . . 7
- Recent Scan Failures . . . 7
- Recent Stale Connection Events . . . 7

When Recovery Is Triggered **8**

Why Missing Interface Does Not Immediately Trigger Recovery **10**

Recovery Sequence **10**
- Soft Stage . . . 11
- Hard Stage . . . 12

Arduino Serial Protocol Use **12**

Serial Port Handling **13**

Cooldown And Locking **13**
- Recovery Cooldowns . . . 14
- Recovery Lock . . . 14

Timeout Handling **15**

Readiness Wait **15**

Spoofed Failure Test **15**

Dry-Run Mode **16**

Systemd Integration **16**

Configuration **17**
Important Settings 17

Installer Behavior **18**

Udev And Stable Serial Naming **19**

Testing **19**

Typical Operational Commands **20**

Current Design Tradeoffs **20**

Mental Model **20**

This document explains the host-side WiFi Watchdog daemon. The goal is to describe what the daemon does, why it exists, and how its pieces work together, without assuming deep familiarity with Linux networking internals.

What Problem The Watchdog Solves

The server depends on a USB WiFi adapter. That adapter can get into a bad state where it is still physically present, but WiFi networking no longer works correctly.

The important detail is that this failure is not always a clean disconnect. A simple "is the USB device present?" check is not enough. The adapter can still show up in Linux while `wpa_supplicant` cannot talk to it properly, scans fail, or DHCP never returns an IPv4 address.

The watchdog handles that by watching the WiFi interface from software and, when it stays unhealthy long enough, asking the Arduino relay controller to briefly cut and restore power to the WiFi stick.

In plain terms:

- Linux notices the WiFi stick is stuck.
- The watchdog sends `RESET` to the Arduino over serial.
- The Arduino pulses the relay.
- The WiFi stick loses power for a short time.
- The stick reconnects as if it had been unplugged and plugged back in.
- The watchdog restarts the network services in a controlled order.

Files Involved

The watchdog implementation is split into a few small files.

File	Purpose
`tools/wifi-watchdog.py`	Main Python daemon. Performs health checks, triggers recovery, talks to the Arduino.
`systemd/wifi-watchdog.service`	Systemd unit that runs the daemon continuously.
`systemd/wifi-watchdog.env.example`	Default configuration values used by the systemd unit.
`scripts/install-watchdog-service.sh`	Installer/updater for the daemon and service.
`test/test_watchdog.py`	Hardware-free tests for the daemon logic.
`udev/99-stick-killer.rules`	Example udev rule for stable `/dev/stick-killer` naming.

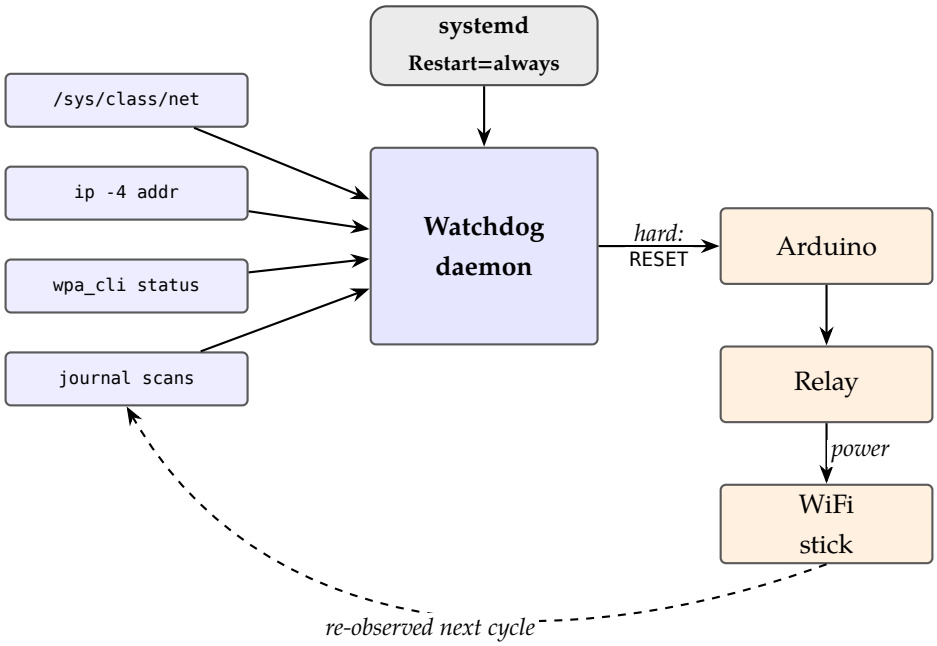
The firmware side is separate. It only needs to understand simple serial commands such as `PING`, `STATUS`, and `RESET`.

High-Level Architecture

The daemon has four main responsibilities:

1. Periodically inspect the WiFi interface.
2. Decide whether the interface is healthy, temporarily unhealthy, or stuck.
3. Run a staged recovery when the stick is stuck — a software-only *soft* attempt first, escalating to a *hard* relay power-cycle only if the soft attempt fails.
4. Avoid repeated or overlapping recoveries.

At a glance, the daemon reads several health signals, decides, and — only when a soft attempt has already failed — drives the relay path the firmware exposes (the soft stage never touches the relay):

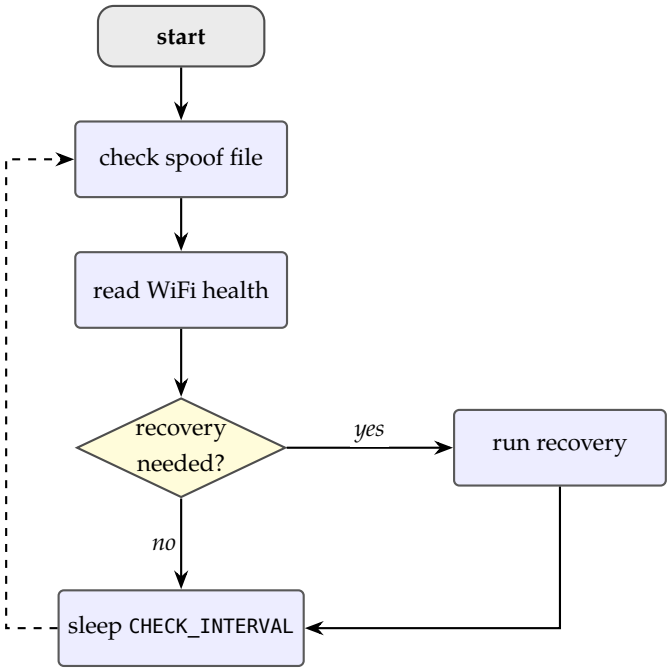


The relay path shown here is the *hard* stage. The *soft* stage runs entirely inside the daemon (restarting the WiFi software) and never reaches the Arduino — see Recovery Sequence.

The main loop is intentionally simple:

```
repeat forever:
    check if a spoofed failure was requested
    read current WiFi health
    decide whether recovery is needed
    if needed, run recovery
    sleep for CHECK_INTERVAL seconds
```

As a flow:



This keeps the daemon predictable. It does not try to be a full network manager. It watches one known interface and performs one known recovery action.

Health Model

The daemon summarizes the current WiFi state in a `Health` object.

```
@dataclass
class Health:
    iface_exists: bool
    has_ipv4: bool
    wpa_state: str | None
    scan_failures: int
    stale_events: int = 0

    @property
    def connected(self) -> bool:
        return self.wpa_state == "COMPLETED" and self.has_ipv4
```

Each field answers one practical question.

Field	Meaning
`iface_exists`	Does Linux currently expose the configured network interface under `/sys/class/net`?
`has_ipv4`	Does the interface have a global IPv4 address?

Field	Meaning
`wpa_state`	What does `wpa_cli status` report for the interface?
`scan_failures`	How many recent `CTRL-EVENT-SCAN-FAILED ret=-1` lines appeared in the relevant `wpa_supplicant` journal?
`stale_events`	How many recent WPA churn events (disconnects, association rejects, EAP failures, SSID temp-disables, ignore-list backoff) appeared in the journal?

The daemon treats the interface as fully connected only when both of these are true:

- `wpa_state == "COMPLETED"`
- `has_ipv4 == True`

This is deliberately stricter than just asking whether the interface exists. The WiFi stick can be visible to the kernel while still not being useful as a network connection.

How Health Is Read

Interface Presence

The daemon checks:

```
/sys/class/net/<interface>
```

For the current deployment, the interface is:

```
wlx40a5efd86cc5
```

If that path exists, Linux currently knows about the interface.

IPv4 Address

The daemon runs:

```
ip -o -4 addr show dev <interface> scope global
```

If this command returns at least one global IPv4 address, `has_ipv4` is true.

This matters because WPA association alone is not enough. The bridge and routing setup need a usable IP layer as well.

WPA State

The daemon runs:

```
wpa_cli -i <interface> status
```

Then it looks for a line like:

```
wpa_state=COMPLETED
```

If `wpa_cli` cannot connect to the control socket, the daemon records `wpa_state=None`. That is considered unhealthy, but not immediately fatal. The daemon waits for the configured unhealthy timeout before power-cycling the stick.

This handles logs like:

```
Failed to connect to non-global ctrl_ifname: wlx40a5efd86cc5 error: No such file or directory
```

That message means the interface may still exist, but the WPA control path is not usable.

Recent Scan Failures

The daemon looks at the recent journal for:

```
wpa_supplicant@<interface>.service
```

It counts occurrences of:

```
CTRL-EVENT-SCAN-FAILED ret=-1
```

The scan-failure count is useful for the original observed failure mode, where the adapter repeatedly failed scans while the interface was otherwise still present.

Recent Stale Connection Events

Scan failures are not the only churn signal. The adapter can also flap: it briefly reaches `wpa_state=COMPLETED`, drops, retries, and repeats. A single "is it connected right now?" snapshot can miss this, because it may happen to look during one of the brief good moments.

To catch that, the daemon counts recent WPA churn events in the same journal window. It looks for lines containing any of:

```
CTRL-EVENT-DISCONNECTED
CTRL-EVENT-ASSOC-REJECT
CTRL-EVENT-EAP-FAILURE
CTRL-EVENT-SSID-TEMP-DISABLED
ignore list count
```

If `stale_events` reaches `MIN_STALE_EVENTS`, the interface is treated as wedged even if it currently looks connected.

To avoid acting on the *same* churn twice, a successful recovery records a timestamp in:

```
<STATE_DIR>/stale-ignore-before
```

Journal lines at or before that timestamp are ignored when counting stale events, so log history from before the last recovery cannot re-trigger another one.

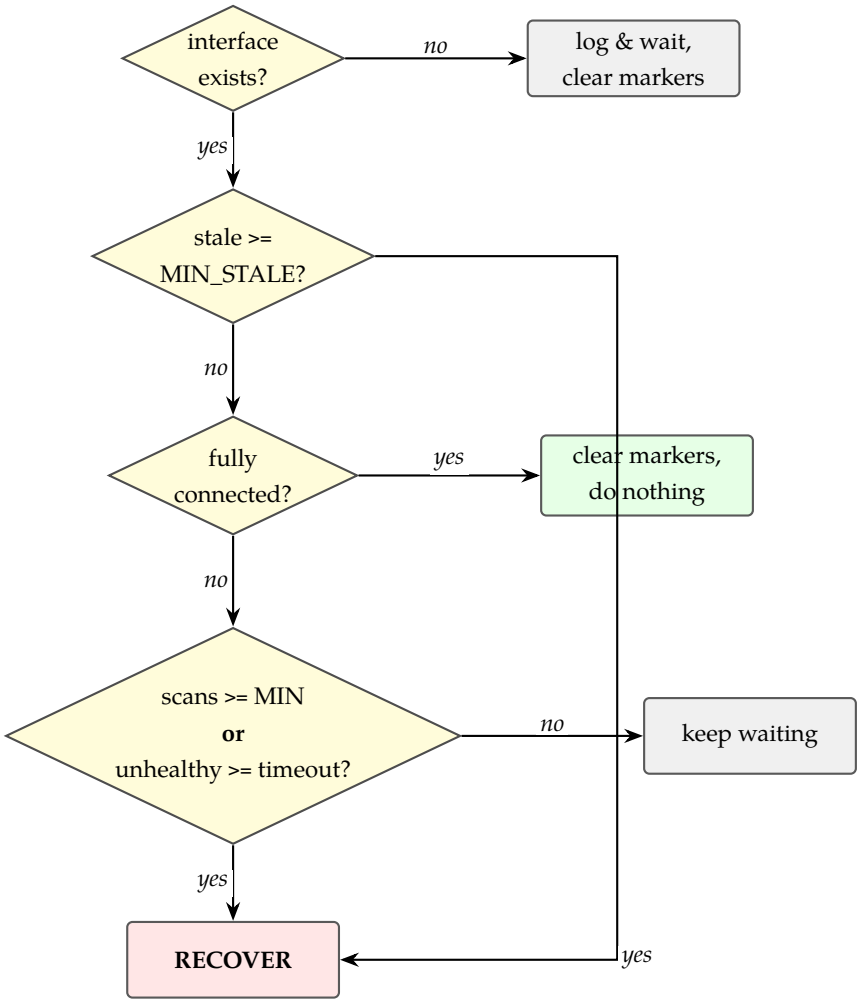
When Recovery Is Triggered

The daemon does not power-cycle immediately on every small problem. WiFi can be temporarily unhealthy during boot, DHCP renewal, service restarts, or normal association.

Recovery is triggered when the interface exists and one of these is true:

- recent stale connection events reach `MIN_STALE_EVENTS` — this fires even if the interface momentarily looks connected, because flapping is itself the failure
- the interface is not fully connected **and** recent scan failures reach `MIN_SCAN_FAILURES`
- the interface is not fully connected **and** has stayed that way for at least `UNHEALTHY_TIMEOUT`

The full decision, per check cycle, is:



Whichever branch fires, the action is the *staged* recovery described below — the daemon decides soft-versus-hard separately, based on whether a previous soft attempt already failed.

The sustained-unhealthy timer is stored in:

```
<STATE_DIR>/unhealthy-since
```

With the default state directory, that is:

```
/run/wifi-watchdog/unhealthy-since
```

The timer is cleared when the interface becomes fully connected again.

This design catches both known failure shapes:

- scan-failure storms
- no IPv4 and no usable `wpa_cli` status, even with zero scan failures

Why Missing Interface Does Not Immediately Trigger Recovery

If the network interface does not exist, the daemon logs that fact but does not immediately pulse the relay.

That is intentional. A missing interface can happen briefly while USB is being renumerated, during boot, or while the device is already recovering. Pulsing the relay at that moment can make startup noisier instead of better.

The current policy is:

- interface exists but stays unhealthy: recover
- interface is fully connected: do nothing
- interface is missing: log it and wait

Recovery Sequence

When the daemon decides the stick is stuck, it runs a **staged** recovery. The guiding idea is to try the cheapest, least disruptive fix first and only reach for the relay if software alone cannot fix it.

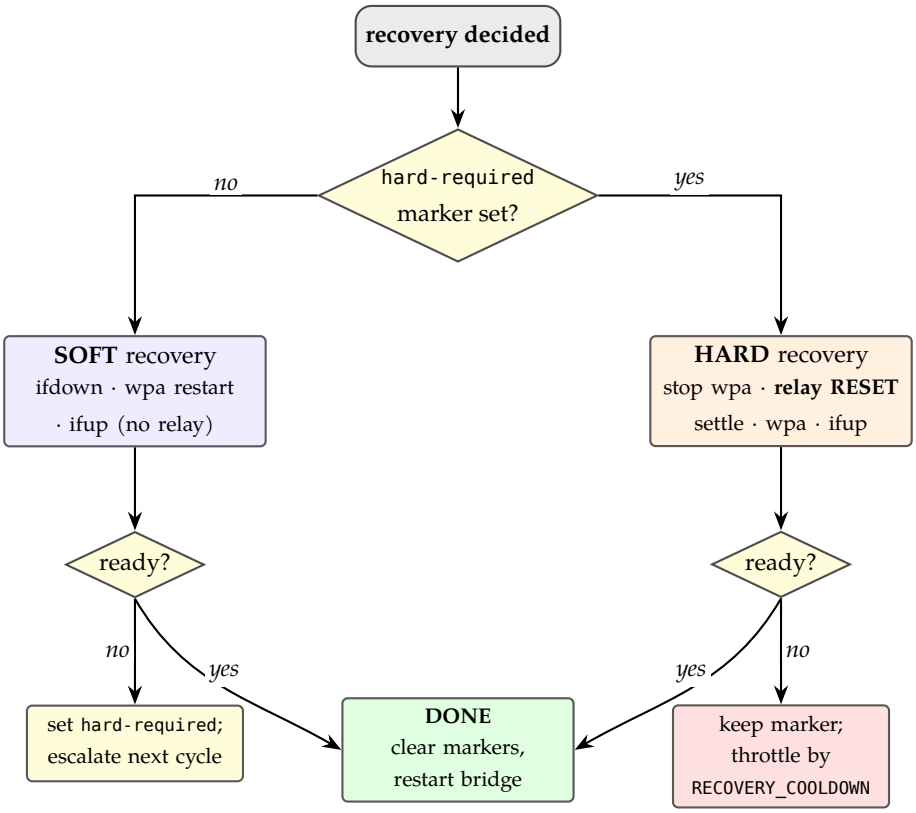
There are two stages:

- a **soft** stage that restarts the WiFi software but never pulses the relay
- a **hard** stage that power-cycles the stick through the Arduino relay

A persistent marker decides which stage runs:

```
<STATE_DIR>/hard-required
```

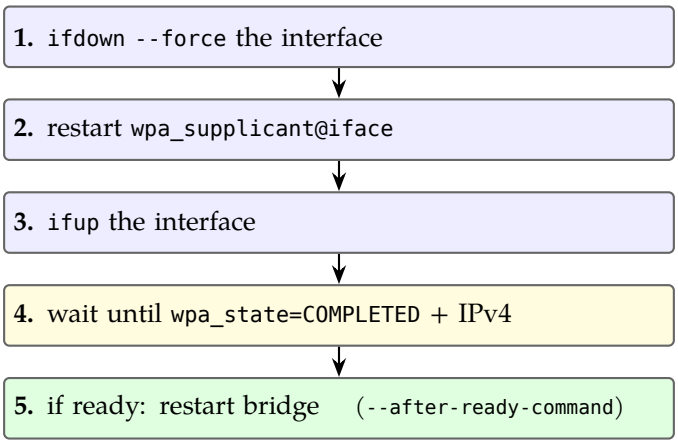
If that marker is absent, the next recovery is soft. If a soft attempt fails to restore the link, the daemon writes the marker, so the *next* check escalates to hard. A successful recovery of either kind clears the marker.



A manual spoof (`force-dead`) intentionally skips the gate and runs the hard stage directly — see Spoofed Failure Test.

Soft Stage

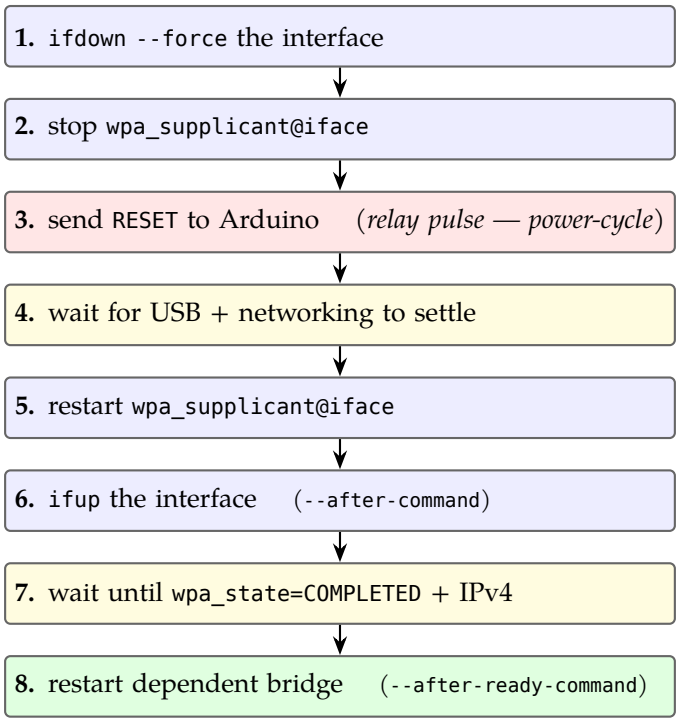
The soft stage is configured with `--soft-command` entries and is throttled by `SOFT_RECOVERY_COOLDOWN` (default 60 s):



If the soft stage restores the link, **no relay pulse ever happens** and the markers are cleared. If it does not, the daemon sets `hard-required`.

Hard Stage

The hard stage runs `--before-command`, pulses the relay, waits for the USB stack to settle, then runs `--after-command` and the readiness wait. It is throttled by `RECOVERY_COOLDOWN` (default 600 s):



The order matters.

The old approach restarted `eduroambridge.service` immediately after `ifup`. That was too early in practice. `ifup` may return or time out while association or DHCP is still in progress. The bridge service should only restart once the WiFi side is actually usable.

Both stages therefore share two types of deferred command:

Command Type	Meaning
`--after-command`	Run after the relay pulse and settle delay (hard stage). Used to bring WiFi back up.
`--after-ready-command`	Run only after `wpa_state=COMPLETED` and IPv4 are present. Used for dependent services such as the bridge. Applies to both stages.

Arduino Serial Protocol Use

During recovery, the daemon opens the stable serial device:

```
/dev/stick-killer
```

Then it speaks the firmware protocol:

```
PING
STATUS
RESET
```

The expected exchange is:

```
PING    -> PONG
STATUS  -> OK READY
RESET   -> OK RESETTING
           OK DONE
```

The daemon checks these responses because blindly toggling a serial line would be risky. If the Arduino is not reachable, not ready, or returns an unexpected response, the daemon raises an error instead of pretending recovery happened.

Serial Port Handling

The `SerialRelay` helper uses low-level POSIX calls instead of pyserial. That keeps the daemon dependency-free.

It does the following:

- opens the serial device in non-blocking mode
- configures the baud rate and raw terminal mode
- flushes stale input and output
- waits briefly after opening the device
- sends one ASCII command per line
- reads one response line at a time with a timeout

The open delay is useful because many Arduino boards reset when the serial port is opened. The daemon waits before sending commands so the firmware has time to boot and print any startup text.

Cooldown And Locking

The daemon has two protections against repeated recoveries.

Recovery Cooldowns

The two stages have independent cooldowns, so a cheap soft retry is not blocked by the much longer hard-relay cooldown:

Stage	Cooldown setting	Default	Timestamp file
Soft	`SOFT_RECOVERY_COOLDOWN`	60 s	`<STATE_DIR>/last-soft-recovery`
Hard	`RECOVERY_COOLDOWN`	600 s	`<STATE_DIR>/last-hard-recovery`

Each stage checks its own timestamp file before running and refuses to start if the cooldown has not elapsed. A successful recovery of either kind also updates `<STATE_DIR>/last-recovery` for a combined "last did anything" marker.

The long hard cooldown matters most after a *failed* hard recovery: the `hard-required` marker stays set, but future hard attempts are spaced at least `RECOVERY_COOLDOWN` apart so the relay and USB stick are not hammered when a power-cycle is not helping.

The full set of runtime state files is:

File	Purpose
`unhealthy-since`	When the interface first became not-fully-connected.
`hard-required`	Present once a soft attempt has failed; forces the hard stage.
`last-soft-recovery`	Timestamp of the last soft attempt (soft cooldown).
`last-hard-recovery`	Timestamp of the last hard attempt (hard cooldown).
`last-recovery`	Timestamp of the last successful recovery of either kind.
`stale-ignore-before`	Journal cutoff so old churn does not re-trigger recovery.
`recovery.lock`	Lock held during any recovery (see below).

Recovery Lock

The daemon also uses a file lock:

```
<STATE_DIR>/recovery.lock
```

This prevents overlapping recoveries if two invocations are ever run at the same time. In normal systemd operation there should only be one daemon process, but the lock makes manual tests safer.

Timeout Handling

The daemon runs helper commands through one wrapper. That wrapper applies `COMMAND_TIMEOUT`, captures stdout and stderr, and logs failures.

For non-critical helper commands, failures are warnings rather than fatal daemon crashes. This is important because commands such as `ifup` can hang or time out during a real network recovery. A timed-out helper command should not kill the watchdog process.

The default is:

```
COMMAND_TIMEOUT=120
```

If a command times out, the daemon logs a warning and continues.

Readiness Wait

Before running after-ready commands, the daemon waits up to:

```
READY_TIMEOUT=90
```

It polls every:

```
READY_POLL_INTERVAL=2
```

Each poll rereads the health state. The interface is ready only when:

```
wpa_state=COMPLETED
has_ipv4=True
```

If readiness does not happen in time, the daemon skips the after-ready commands and logs a warning. That prevents dependent services from being restarted into a known-bad network state.

Spoofed Failure Test

The daemon supports a deliberate test trigger:

```
sudo touch /run/wifi-watchdog/force-dead
```

On the next check, the daemon deletes that file and runs recovery. Unlike an automatic failure, the spoof **skips the soft stage and forces the hard relay recovery directly**, because the whole point of the manual trigger is to exercise the relay path end to end.

This is useful because it tests:

- systemd integration
- serial communication with the Arduino
- relay pulse behavior
- service restart commands
- readiness wait
- cooldown behavior

It is not a dry run. It intentionally causes a real relay pulse.

Dry-Run Mode

For a no-hardware test, the daemon has:

```
--dry-run
```

In dry-run mode it logs the commands and relay pulse it would perform, but does not actually run them.

That mode is useful for checking command formatting and configuration without power-cycling the WiFi stick.

Systemd Integration

The systemd unit is:

```
systemd/wifi-watchdog.service
```

It installs to:

```
/etc/systemd/system/wifi-watchdog.service
```

The unit runs the Python daemon as a normal long-running service:

```
[Service]
Type=simple
EnvironmentFile=/etc/wifi-watchdog.env
ExecStart=/usr/local/sbin/wifi-watchdog.py ...
```

```
Restart=always
RestartSec=10
```

`Restart=always` means systemd will restart the watchdog if the Python process exits unexpectedly. The daemon itself is written to avoid crashing on ordinary network command failures, but systemd still provides a final safety net.

Configuration

The installed configuration lives at:

```
/etc/wifi-watchdog.env
```

The example file contains:

```
WIFI_STICK_IFACE=wlx40a5efd86cc5
RELAY_SERIAL=/dev/stick-killer
CHECK_INTERVAL=15
FAILURE_WINDOW=90
MIN_SCAN_FAILURES=10
MIN_STALE_EVENTS=6
UNHEALTHY_TIMEOUT=90
SOFT_RECOVERY_COOLDOWN=60
RECOVERY_COOLDOWN=600
COMMAND_TIMEOUT=120
READY_TIMEOUT=90
READY_POLL_INTERVAL=2
SPOOF_DEAD_FILE=/run/wifi-watchdog/force-dead
STATE_DIR=/run/wifi-watchdog
```

Important Settings

Setting	Meaning
`WIFI_STICK_IFACE`	The WiFi interface to monitor.
`RELAY_SERIAL`	Stable serial path for the Arduino relay controller.
`CHECK_INTERVAL`	Seconds between health checks.
`FAILURE_WINDOW`	Journal time window used for scan-failure counting.
`MIN_SCAN_FAILURES`	Scan failures needed to trigger immediate recovery while unhealthy.

Setting	Meaning
`MIN_STALE_EVENTS`	Recent WPA disconnect/reject/failure events needed to trigger stale-connection recovery.
`UNHEALTHY_TIMEOUT`	Seconds the interface may remain not fully connected before recovery.
`SOFT_RECOVERY_COOLDOWN`	Minimum seconds between soft recovery attempts.
`RECOVERY_COOLDOWN`	Minimum seconds between hard relay recovery attempts.
`COMMAND_TIMEOUT`	Timeout for helper commands such as `ifup`.
`READY_TIMEOUT`	Maximum time to wait for WiFi to become ready after reset.
`READY_POLL_INTERVAL`	Seconds between readiness checks.
`SPOOF_DEAD_FILE`	File path used to trigger a forced test recovery.
`STATE_DIR`	Runtime state directory for locks and timestamps.

Installer Behavior

The installer script is:

```
scripts/install-watchdog-service.sh
```

It installs or updates:

```
/usr/local/sbin/wifi-watchdog.py
/etc/systemd/system/wifi-watchdog.service
/etc/wifi-watchdog.env
/usr/local/share/wifi-watchdog/README.md
```

On update, it preserves existing values in `/etc/wifi-watchdog.env`. If a new setting has been added to the example env file, the installer adds it using the default value.

Before rewriting the env file, it creates a timestamped backup:

```
/etc/wifi-watchdog.env.bak.YYYYMMDD-HHMMSS
```

If the service is already running, the installer restarts it after installing the new files. If it is not running, the installer does not start it unless `--start` is passed.

Udev And Stable Serial Naming

The Arduino uses a CH341 USB serial adapter. This adapter does not expose a unique serial number, so a stable device name must be tied to the physical USB path.

The target stable path is:

```
/dev/stick-killer
```

The included udev rule matches:

```
idVendor=1a86
idProduct=7523
USB path=3-1.5.1.2
```

This means the mapping is stable as long as the Arduino remains plugged into the same USB port. If the Arduino moves to a different port, the udev rule must be updated.

Testing

The watchdog has hardware-free tests in:

```
test/test_watchdog.py
```

They use a fake watchdog subclass instead of touching the real system. The tests verify:

- after-ready commands wait for IPv4
- after-ready commands are skipped when the interface never becomes ready
- sustained unhealthy state triggers recovery even without scan failures
- a connected interface clears the unhealthy marker
- a fresh failure routes to the soft stage first (no relay pulse)
- a failed soft attempt escalates the next check to the hard stage
- a successful soft or hard recovery does not latch `hard-required`
- a successful soft recovery does not also run the hard stage
- a failed hard recovery keeps later hard attempts cooldown-limited

Run them with:

```
python3 test/test_watchdog.py
```

The daemon can also be syntax-checked with:

```
python3 -m py_compile tools/wifi-watchdog.py
```

The installer can be checked with:

```
bash -n scripts/install-watchdog-service.sh
```

Typical Operational Commands

Install or update the service:

```
cd /home/tudo/Downloads/wifi-stick-killer
sudo ./scripts/install-watchdog-service.sh
```

Watch logs:

```
sudo journalctl -u wifi-watchdog.service -f
```

Check service status:

```
sudo systemctl status wifi-watchdog.service --no-pager
```

Trigger a real test recovery:

```
sudo touch /run/wifi-watchdog/force-dead
```

Check the Arduino serial symlink:

```
ls -l /dev/stick-killer
```

Current Design Tradeoffs

The daemon is deliberately conservative in a few places.

It does not try to repair every possible network failure. It focuses on the known WiFi stick failure modes and one specific recovery action.

It does not immediately recover when the interface disappears. That avoids fighting normal USB renumeration and startup timing.

It does not require scan failures anymore. Scan failures are one strong signal, but sustained lack of WPA completion and IPv4 is also enough.

It does not restart dependent services until WiFi is actually ready. This avoids starting the bridge while DHCP or WPA association is still incomplete.

It keeps external dependencies minimal. The Python script uses only the standard library and standard Linux commands already expected on the server.

Mental Model

The daemon is best thought of as a patient supervisor.

It does not manage WiFi minute by minute. It watches for a small set of symptoms that mean the USB stick has probably wedged. If the symptoms persist, it first tries the gentle fix — restarting the WiFi software — and only asks the Arduino to power-cycle the stick when that gentle fix has already failed.

After whichever stage restores the link, it brings the network stack back in a controlled order and only restarts dependent bridge logic after the WiFi interface is genuinely usable again.